

COMPSCI5079 (M)

Cryptography & Secure Development

Dr. Nguyen Truong

Lecturer (Assistant Professor), SoCS, University of Glasgow

Email: Nguyen.Truong@Glasgow.ac.uk

Office: SAWB 221C, Sir Alwyn Williams Building, 18 Lilybank Gardens





Course Structure

The course structure consists of 5 main sessions:

1. Fundamental Foundation for Cryptography
2. Symmetric Encryption schemes
 - Traditional Encryption
 - Modern Encryption schemes
 - Message digests, random numbers, multiple-key encryption
3. Public-key Encryption/Asymmetric key encryption
 - Diffie-Hellman, RSA
 - Elliptic Curve Cryptography
 - Crypto-currencies (e.g., Bitcoin, Ethereum) and Blockchain
4. Coding and Security
5. Secure Development

Lecture 1:

Fundamental Foundation for Cryptography





Lecture Outline

1. Cryptography Overview & Terminology

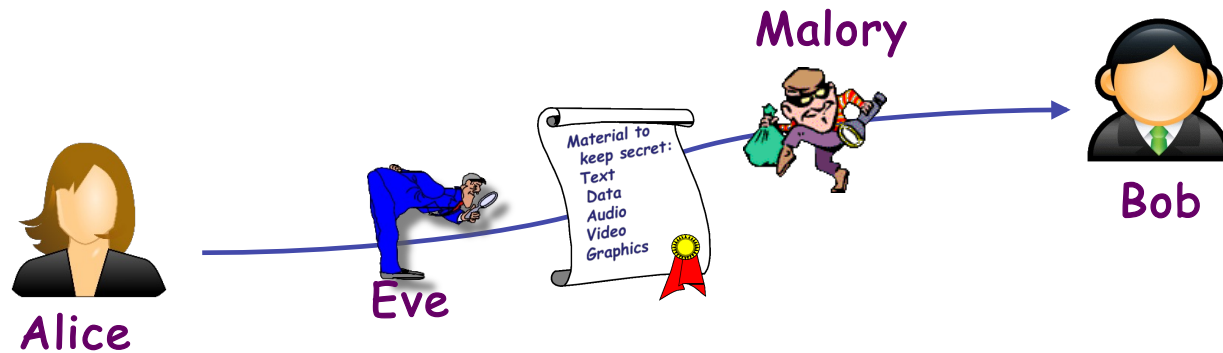
2. Mathematical Foundation

- Integer mod N
- Polynomial Arithmetic

3. Information Theory



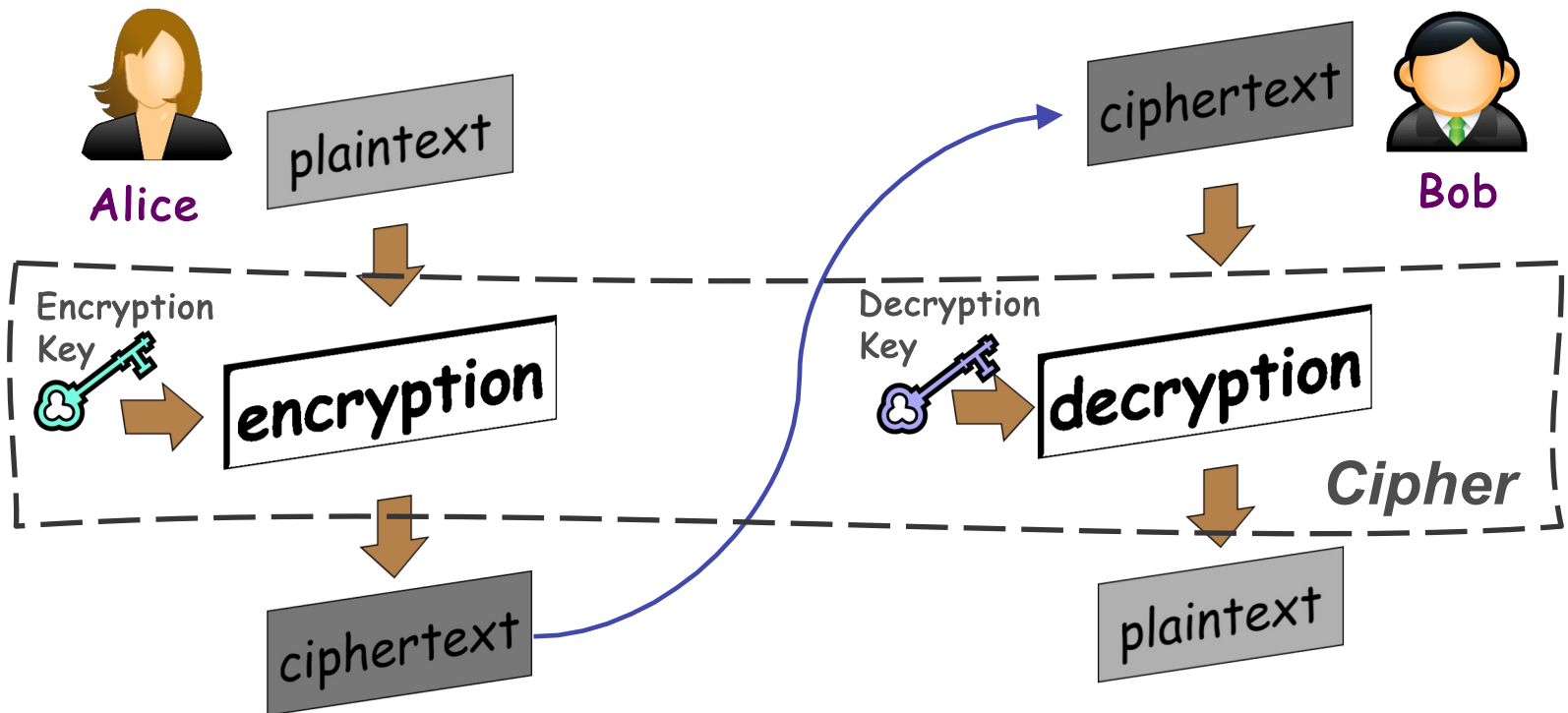
Cryptography Overview



- **Problem:** Alice and Bob would like to exchange messages (over a public network) without revealing to anyone but the intended recipient.
- **Eve and Malory** are adversaries
- **Solution:** Cryptography



How does Cryptography work?





Terminology

- **Plaintext:** original message needs to be transmitted which is information that can be directly read by humans or a machine.
- **Ciphertext:** the encrypted data which is "unreadable"
- **Cipher:** the mathematics (or algorithm) responsible for turning plaintext into ciphertext and reverting ciphertext to plaintext.
 - *Stream Cipher & Block Cipher*
- **Encryption:** the process of converting plaintext to ciphertext
- **Decryption:** the process of reverting ciphertext to plaintext
- **Secret Keys:** an additional input for Encryption or Decryption in a way that Decryption can only be performed by the proper key.



Plaintext as Integers

Plaintext: original message (e.g., letters, bitstring, etc.) that needs to be securely transmitted

We note that:

- Letters can be represented as integers.
 - Use the **ASCII** or similar code.
- A series of bits (a bitstring) can be represented as a series of integers.
 - Split it into chunks and treat each chunk as a binary number.
- Thus, text can always be represented as a series of integers.

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0		[NULL]	32	20	[SPACE]	64	40	@	96	60	
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]



Encryption & Decryption: First Intuition

Cipher: How to turn plaintext into ciphertext, and turn it back?

A simple example:

- Let us assume that our alphabet consists of 26 letters and 3 other characters: space (-), and .
 - ✓ We can represent these 29 characters by the numbers 0 .. 28.
- A simple way of **encrypting plaintext** is to multiply the letter value by a number between 2 and 28, the **secret key**.
 - ✓ This will normally produce a number larger than 28, making it hard to write down the ciphertext.
 - ✓ We can divide the ciphertext by 29 and keep the remainder, which will be in the range [0 .. 28] and can be represented by one of our characters.
- We can **decrypt** the ciphertext by dividing it by the secret key, which will not normally produce an integer!



Course Assessment

Example:

- Encrypt the plaintext: "I LOVE YOU, BOB."
 - Our alphabet characters A B C ... Z (-) (,) and (.) are represented by 0 1 2 ... 25 26 27 28, respectively.
 - Our secret key is 10
 - I $\rightarrow$ Remainder of $(8 \times 10) \% 29 = 22 \rightarrow$ W
 - (-) $\rightarrow$ Remainder of $(27 \times 10) \% 29 = 9 \rightarrow$ J
 - L $\rightarrow$ Remainder of $(11 \times 10) \% 29 = 23 \rightarrow$ X

- **Question:** How to decrypt the ciphertext if we know the secret key (i.e., 10): WJX...
 - W $\rightarrow$ 22 $\rightarrow$?
 - J $\rightarrow$ 9 $\rightarrow$?
 - X $\rightarrow$ 23 $\rightarrow$?



English Alphabet in Order

A	B	C	D	E	F	G	H	I	J
1	2	3	4	5	6	7	8	9	10
K	L	M	N	O	P	Q	R	S	T
11	12	13	14	15	16	17	18	19	20
U	V	W	X	Y	Z				
21	22	23	24	25	26				



Mathematical Foundation

2. Mathematical Foundation

- Integer mod N
- Polynomial Arithmetic



Integer modulo n

Integer mod n :

- Let us only use some of the integers, not all of them.
 - All the integers between $(0 \dots n-1)$ for some n
 - n can be very large, say 100 digits!
- Any integer outside the range $(0 \dots n-1)$ can be converted to an integer inside the range by dividing by n and keeping the remainder
 - Like our simple example.
- These are called "integers mod n "
 - For example, "integers mod 7" contain the following 7 numbers:
 - $\{ 0, 1, 2, 3, 4, 5, 6 \}$
 - 10, 17, 24 are all the same as 3
 - -1 is the same as 6



Inverses

- If a is an "integer mod n " and x is its inverse, then they satisfy the equation:

$$ax \equiv 1 \text{ mod } n$$

- Any number multiplied by its inverse is 1
- We know a and n , and we want to find x
- $\equiv$ means "equivalent" after the modulus operation.
 - If $a=3$, $x=4$, $n=11$ then $ax = 12$, which is equivalent to $(\equiv) 1$
- x must also be an integer, but modular arithmetic makes this possible.

I will often use % to mean mod (it is used for mod in programming).



Example: Integer mod 7

Inverses in Integers mod 7 number system

- 0 does not have an inverse.
- By convention, the inverse of 0 is 0.
- The inverse of 1 is 1.
- The inverse of 6 is 6
- $6 \cdot 6 = 36 \equiv 1 \pmod{7}$ which satisfies the equation:
- $6 \equiv -1$ and $-1 \cdot -1 = 1$
- The inverse of 2 is 4 (and inverse of 4 is 2):
- $2 \cdot 4 = 8 \equiv 1 \pmod{7}$
- The inverse of 3 is 5 (and inverse of 5 is 3):
- $3 \cdot 5 = 15 \equiv 1 \pmod{7}$



Example: Integer mod 4

Considering Integers mod 4 number system

- Possible values are $\{0, 1, 2, 3\}$
- The inverse of 1 is 1
- The inverse of 3 is 3
 - $3 * 3 = 9 \equiv 1 \pmod{4}$.
- 2 does not have an inverse (in "integers mod 4" number system). Why?
 - 2 times anything is always an even number.
 - Dividing by 4 always leaves an even remainder.
 - Therefore, the remainder cannot be 1.
- Modular arithmetic does not guarantee that all numbers have inverses.



Existence of Inverses

Question: When will a number have an Inverse?

- Let a be a number in the "integer mod n " number system.
- a will have an inverse if and only if $\gcd(a, n) = 1$
 - $\gcd$ is the "greatest common divisor" or "highest common factor".
 - This condition means that a and n do not have any common factors.
 - $\gcd(2, 4) = 2$ and so 2 does not have an inverse in the integers mod 4 number system.
 - The proof is beyond the scope of this course.
- If n is a prime number, then all numbers a , except 0, will have an inverse.
 - It was lucky that I chose $n = 29$ for my example system!



How to find Inverses?

- The previous slide shows when an inverse exists but not how to calculate it.
 - An algorithm will follow.
- First, we need a general theorem that is applicable to all calculations with "integer mod n " number system.



The Homomorphism Theorem

- We can perform the modulus operation whenever it is convenient.
- The final answer will always be the same.
- $(a \otimes b) \% n = ((a \% n) \otimes (b \% n)) \% n$
 - where $\otimes$ is +, - or *
 - $\%$ is the modulus operator.
- The proof is beyond the scope of the course.

- For example, calculate $20 * 11$ with "integers mod 7".
 - $(20 * 11) \% 7 = 220 \% 7 = 3$ (after some long division)
 - $(20 \% 7) * (11 \% 7) \% 7 = (6 * 4) \% 7 = 24 \% 7 = 3$
- The numbers are smaller in the second calculation.



Example: Divisible by 9

- A number is **divisible by 9** if we add up all the digits and that sum is divisible by 9.
- For instance, **1234566** is divisible by 9 as **$(1+2+3+4+5+6+6)=27$** which is divisible by 9.
- We can use the homomorphism theorem to prove it.

If a number n has digits a_i , then a_i represents the powers of 10, so:

$n = \sum a_i \times 10^i$, where $\sum$ means sum over all values of i .

$$\begin{aligned}\text{Now: } n \% 9 &= (\sum a_i \times 10^i) \% 9 \\ &= \sum (a_i \% 9) \times (10 \% 9)^i\end{aligned}$$

$$\begin{aligned}\text{Now: } (10 \% 9 = 1) \text{ and } (1^i = 1) \text{ for all values of } i, \text{ thus:} \\ &= (\sum a_i) \% 9\end{aligned}$$

- Similar reasoning can be used to prove the rule for division by 11.



Example: Calculate $3^5 \% 7$

- Use a repeated squaring algorithm.

$$3^2 = 9$$

$$3^4 = 9^2 = 81$$

$$3^5 = 3 * 3^4 = 3 * 81 = 243$$

$$\rightarrow 3^5 \% 7 = 243 \% 7 = 5$$

- Alternatively, we can use the homomorphism theorem to do % at each step

$$3^2 \% 7 = 9 \% 7 = 2$$

$$3^4 \% 7 = 2^2 \% 7 = 4 \% 7 = 4$$

$$\rightarrow 3^5 \% 7 = (3 * 4) \% 7 = 12 \% 7 = 5$$

- The second approach is preferable because the numbers stay small.
- Numbers are typically very large in cryptography.
 - Calculate $(100 \text{ digit})^{100 \text{ digit}} \% 100 \text{ digit}!$



Inverse Algorithm and Division

- The inverse algorithm solves the equation (find x , given a and n) that satisfies:

$$ax \equiv 1 \pmod{n}$$

The inverse of a is x

- This also gives us a way to do division:

$$a / b \pmod{n} \equiv a * (1/b) \pmod{n}$$

- Dividing by b is the same as multiplying by $1/b$



Finding Inverse: Algorithm

1. The following equation is trivially true:

$$n * x \equiv n \pmod{n} \rightarrow E1$$

2. We want to solve:

$$a * x \equiv 1 \pmod{n} \rightarrow E2$$

3. We subtract the two equations to get a third equation E3:

$$E1 - E2 \rightarrow E3$$

If the number on the right is negative, add n . This is allowed with "integers mod n "

4. Discard the equation with the largest number in front of x , and repeat.

For instance: $E2 - E3 \rightarrow E4$; $E3 - E4 \rightarrow E5$..

5. Stop when the number in front of x is 1



Finding Inverse: Algorithm

Note that:

- If a does not have an inverse, then the number in front of x will never be 1.
- The number in front of x will always be positive and get smaller, so this algorithm will always finish.



Finding Inverse: Example

➤ Calculate the inverse x of $8 \bmod 13$

($ax \equiv 1 \pmod{n}$): $a = 8$, $n = 13$; find x)

$$13x \equiv 13 \pmod{13}$$

Step1: E1: always true

$$8x \equiv 1 \pmod{13}$$

Step2: E2: starting equation

$$5x \equiv 12 \pmod{13}$$

Step3: E3: $E1 - E2$

$$3x \equiv -11 \equiv 2 \pmod{13}$$

Step4: E4: $E2 - E3$

$$2x \equiv 10 \pmod{13}$$

Step4: E5: $E3 - E4$ (repeat)

$$x \equiv -8 \equiv 5 \pmod{13}$$

Step5: E6: $E4 - E5$ (repeat & stop)

Check: with $x = 5$ we have:

$$8 * 5 = 40 = 3 * 13 + 1 \rightarrow 8 * 5 \equiv 1 \pmod{13}$$

In other words: 5 is inverse of 8 *mod* 13



Encryption and Decryption:
An example using
Integer mod n Number system



Example: Encryption

- The letters **a-z** have the values **0..25**, with **_**=26, **,**=27 and **.**=28.
- Let the plaintext be **"secret _ message"**

1. Turning the plaintext characters into numbers:

`<18,4,2,17,4,19,26,12,4,18,18,0,6,4>`

2. Choose secret key = 5 (an arbitrary choice); Multiply each letter by the secret key; then do "integer mod 29".

3. The ciphertext in numbers is:

`<3,20,10,27,20,8,14,2,20,3,3,0,1,20>`

4. Turning the numbers back into letters gives us the ciphertext in letters:

`"duk.uiocuddabu"`

Note that repeated **s** and **e** also show up in the ciphertext as repeated **d** and **u**, a weakness of this cipher.



Example: Decryption

- The secret key = 5. The “decryption key” x is the “inverse of $5 \bmod 29$ ”. Let do the algorithm:

$$29x = 29$$

$$5x = 1$$

$$24x = 28$$

$$19x = 27$$

$$14x = 26$$

$$9x = 25$$

$$4x = 24$$

$$x = -23 = 6$$

E1: always true

E2: the equation to solve

$$E3 = E1 - E2$$

- Check: $5 * 6 = 30 = 29 + 1$.



Example: Decryption

- We have secret key = 5, decryption key = 6, the modulus $n = 29$, and ciphertext = duk.uiocuddabu
- To decode the ciphertext, start with the numerical version of the ciphertext and multiply by the decryption key, which is 6; then do "integer mod 29".
- Ciphertext = $\langle 3, 20, 10, 27, 20, 8, 14, 2, 20, 3, 3, 0, 1, 20 \rangle$

$$(3 * 6) \% 29 = 18$$

$$(20 * 6) \% 29 = 4$$

$$(10 * 6) \% 29 = 2$$

$$(27 * 6) \% 29 = 17$$

$$(8 * 6) \% 29 = 19$$

...



Example: Decryption

- This recovers the plaintext in numbers:

$\langle 18, 4, 2, 17, 4, 19, 26, 12, 4, 18, 18, 0, 6, 4 \rangle$

- In letters, the plaintext is: secret _ message

We successfully decrypt the original message (i.e., plaintext) from the ciphertext.

Discussion: *From the Decryption side, what information do we need to successfully decrypt the ciphertext? What information should be kept secret, and what can be public?*



Finding Inverse: Fast Algorithm

- Look at the sequence of subtractions in the example when finding the decryption key x
 - Decryption key x is the “inverse of $5 \bmod 29$ ”.
- It is more efficient to find the inverse by division rather than subtraction.
 - The subtraction algorithm is useful for small examples.
 - An even faster binary algorithm is based on subtraction and removing powers of 2.
- Fast algorithms also exist for:
 - Calculating $\gcd$.
 - Calculating exponents: $a^z \bmod n$ which is used in several encryption algorithms.



Quizzes

1. Calculate $5^7 \bmod 11$ by hand using “repeated squaring” and the “homomorphism theorem”. Verify that the calculations would be much harder if you left the $\bmod 11$ calculation to the end.
2. Calculate $1 / 8 \bmod 11$ (the inverse of 8) by hand using the equation subtracting algorithm. Use your result to calculate $5 / 8 \bmod 11$.



Lecture Summary



Lecture Summary

- Text, and indeed any input, can be converted to a series of integers.
 - Using a numbering scheme.
 - Converting to bits, creating blocks of bits, converting each block to an integer.
- Dividing by integers doesn't usually result in an integer.
- Using $\text{integers mod } n$ lets us define integer inverses.
 - Solve equation: $ax \equiv 1 \pmod{n}$
- If the number a and the modulus n have a common factor, then there is no inverse.
 - Choosing n as a *prime* is convenient.
- Solving equation $ax \equiv 1 \pmod{n}$ uses a trick since $n * x \equiv n \pmod{n}$ is always true.
 - Keep subtracting, reducing the number in front of x each time.
- We can take the remainder when dividing by n whenever we want, which reduces the size of numbers in our calculations.